

Εξαμηνιαία Εργασία

Προχωρημένα Θέματα Βάσεων Δεδομένων



Εργαστήριο Βάσεων Γνώσεων και Δεδομένων
Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο

Αλεξάνδρα Καραλή, 03121084
Σοφία Σάββα, 03121189

Νοέμβριος, 2025

Ο κώδικας που υλοποιήθηκε για την επίλυση των ασκήσεων της εργασίας περιέχεται το εξής αποθετήριο:

<https://github.com/ntua-el21189/Advanced-Topics-in-Database-Systems-Project-Grou-34>

Οι μετρήσεις και τα αποτελέσματα των explain εντολών βρίσκονται στο φάκελο results σε .txt αρχεία αποτελεσμάτων για κάθε query.

Ζητούμενο 1

Υλοποίηση με RDDs

Στην υλοποίηση με RDDs χρησιμοποιήθηκε το SparkContext για την ανάγνωση και επεξεργασία των δεδομένων εγκληματικότητας του Los Angeles από δύο αρχεία CSV που βρίσκονται στο S3 και καλύπτουν τις χρονικές περιόδους 2010–2019 και 2020–2025. Τα αρχεία διαβάζονται ως RDDs γραμμών και γίνεται ανάλυση CSV ώστε κάθε εγγραφή να μετατραπεί σε λίστα πεδίων. Από κάθε εγγραφή διατηρούνται μόνο η περιγραφή του εγκλήματος και η ηλικία του θύματος, καθώς αυτά είναι τα απαραίτητα στοιχεία για το συγκεκριμένο ερώτημα. Στη συνέχεια τα δύο RDDs ενοποιούνται με union και αφαιρείται η γραμμή επικεφαλίδας.

Ακολουθεί φιλτράρισμα των εγγραφών ώστε να διατηρηθούν μόνο τα περιστατικά που περιέχουν τον όρο “AGGRAVATED ASSAULT” στην περιγραφή του εγκλήματος. Για την ομαδοποίηση των θυμάτων, ορίζεται μια βοηθητική συνάρτηση η οποία δέχεται ως είσοδο την ηλικία του θύματος (σε μορφή string), τη μετατρέπει σε ακέραιο αριθμό και την αντιστοιχίζει σε μία από τις τέσσερις προκαθορισμένες ηλικιακές ομάδες: παιδιά (0–17), νεαρούς ενήλικες (18–24), ενήλικες (25–64) και ηλικιωμένους (65+).

Η εφαρμογή της συνάρτησης στο RDD γίνεται μέσω του μετασχηματισμού map, όπου κάθε εγγραφή μετατρέπεται σε ένα ζεύγος της μορφής (ηλικιακή ομάδα, 1). Με αυτόν τον τρόπο, κάθε περιστατικό συνεισφέρει μία μονάδα στην αντίστοιχη ηλικιακή ομάδα στην οποία ανήκει το θύμα. Στη συνέχεια, η reduceByKey χρησιμοποιείται για να αθροίσει τις μονάδες ανά ηλικιακή ομάδα, παράγοντας το συνολικό πλήθος περιστατικών για κάθε κατηγορία.

Τέλος, τα αποτελέσματα ταξινομούνται κατά φθίνουσα σειρά με βάση το πλήθος των περιστατικών και εμφανίζονται στην οθόνη. Παράλληλα, καταγράφεται ο συνολικός χρόνος εκτέλεσης της διαδικασίας, χωρίς να λαμβάνεται υπόψη η φόρτωση και η προεπεξεργασία των δεδομένων, ο οποίος ανέρχεται σε 18.58s.

Υλοποίηση με Dataframes API

Στην υλοποίηση με DataFrames, αρχικά ορίζεται ρητά το σχήμα (schema) των δεδομένων εγκληματικότητας, αντιστοιχίζοντας κάθε στήλη του αρχείου CSV στον κατάλληλο τύπο δεδομένων και, στη συνέχεια, τα δύο αρχεία CSV διαβάζονται από το S3 σε δύο ξεχωριστά DataFrames με ενεργοποιημένο το header και με χρήση του schema που ορίστηκε. Από κάθε DataFrame διατηρούνται μόνο τα πεδία που είναι απαραίτητα για το συγκεκριμένο ερώτημα, δηλαδή η περιγραφή του εγκλήματος και η ηλικία του θύματος. Στη συνέχεια, τα δύο DataFrames ενοποιούνται με τη χρήση της `union`, δημιουργώντας ένα ενιαίο σύνολο δεδομένων. Ακολουθεί, όπως και πριν, φιλτράρισμα των εγγραφών ώστε να διατηρηθούν μόνο τα περιστατικά των οποίων η περιγραφή περιέχει τον όρο “AGGRAVATED ASSAULT”.

Για την κατηγοριοποίηση των θυμάτων σε ηλικιακές ομάδες, δημιουργείται μια νέα στήλη με τη χρήση της `withColumn` και της συνάρτησης `when`. Η ηλικία του θύματος ελέγχεται διαδοχικά και αντιστοιχίζεται σε μία από τις τέσσερις ηλικιακές ομάδες που ορίζονται στην εκφώνηση. Είναι σημαντικό να τονιστεί πως η λογική αυτή υλοποιείται πλήρως με ενσωματωμένες συναρτήσεις του DataFrame API, χωρίς τη χρήση UDF.

Στη συνέχεια, τα δεδομένα ομαδοποιούνται ως προς τη νέα στήλη ηλικιακής ομάδας με χρήση της `groupBy`, και η `count` χρησιμοποιείται για την καταμέτρηση των περιστατικών σε κάθε κατηγορία. Τα αποτελέσματα ταξινομούνται κατά φθίνουσα σειρά με βάση το πλήθος των περιστατικών και εμφανίζονται στην οθόνη. Παράλληλα, καταγράφεται ο συνολικός χρόνος εκτέλεσης της υπολογιστικής διαδικασίας, χωρίς να λαμβάνεται υπόψη η φόρτωση και η αρχική προεπεξεργασία των δεδομένων, ο οποίος ανέρχεται στα 8.88s.

Υλοποίηση με Dataframes API και UDF

Η βασική διαφοροποίηση της παρούσας υλοποίησης αφορά τον τρόπο με τον οποίο πραγματοποιείται η αντιστοίχιση της ηλικίας του θύματος σε ηλικιακή ομάδα. Αντί για τη χρήση ενσωματωμένων συναρτήσεων του DataFrame API, ορίζεται μια συνάρτηση σε Python που υλοποιεί τη λογική κατηγοριοποίησης της ηλικίας στις τέσσερις προκαθορισμένες ομάδες. Η συνάρτηση αυτή μετατρέπεται σε User Defined Function (UDF) και εφαρμόζεται στη στήλη της ηλικίας μέσω της `withColumn`, δημιουργώντας μια νέα στήλη που περιέχει την ηλικιακή ομάδα κάθε εγγραφής.

Τα υπόλοιπα στάδια, που περιλαμβάνουν την προεπεξεργασία των δεδομένων, την ομαδοποίηση, τη καταμέτρηση και την ταξινόμηση, είναι ίδια όσα υλοποιήθηκαν στην προηγούμενη προσέγγιση με DataFrames. Σε αυτή την περίπτωση, ο συνολικός χρόνος εκτέλεσης της υπολογιστικής διαδικασίας ήταν 9.15s.

Σύγκριση των τριών υλοποιήσεων

Η σύγκριση των τριών υλοποιήσεων δείχνει ότι η προσέγγιση με DataFrames χωρίς χρήση UDF είναι η πιο αποδοτική, με χρόνο εκτέλεσης 8.88s, γεγονός αναμενόμενο, αφού αξιοποιεί αποκλειστικά συναρτήσεις του Spark SQL και επωφελείται πλήρως από τις βελτιστοποιήσεις του Catalyst Optimizer. Η υλοποίηση με DataFrames και UDF παρουσίασε ελαφρώς αυξημένο χρόνο εκτέλεσης (9.15 s), γεγονός που αποδίδεται στο ότι οι UDFs αντιμετωπίζονται ως «μαύρα κουτιά» από το Spark, με αποτέλεσμα να μην είναι δυνατή η εφαρμογή βελτιστοποιήσεων, ενώ επιπλέον εισάγεται κόστος serialization-deserialization λόγω της μεταφοράς δεδομένων μεταξύ JVM και Python runtime. Τέλος, η υλοποίηση με RDDs ήταν σημαντικά πιο αργή (18.58 s), όπως άλλωστε αναμενόταν, αφού βασίζεται σε χαμηλού επιπέδου μετασχηματισμούς και δεν αξιοποιεί τις βελτιστοποιήσεις του Spark SQL. Αξίζει να σημειωθεί εδώ πως, για τη διασφάλιση δίκαιης σύγκρισης, πριν από κάθε εκτέλεση γινόταν επανεκκίνηση του kernel, ώστε να αποφεύγεται η επίδραση της cache.

Ζητούμενο 2

Ορίζεται όπως έχει αναφερθεί το schema των csv αρχείων που θα χρησιμοποιηθούν και δημιουργείται το crimes Dataframe που περιέχει τα συνολικά εγκλήματα και το REcodes_df dataframe. Διατηρούνται οι απαραίτητες στήλες για την υλοποίηση του ζητουμένου, DATE_OCC και Vict_Descent. Για τη σωστή ομαδοποίηση ανά έτος γίνεται η μετατροπή της στήλης DATE_OCC σε timestamp και προκύπτει το crimes_df_final.

Υλοποίηση με Dataframe API

Για τον υπολογισμό του ζητουμένου, το DataFrame crimes_df_final ομαδοποιείται ως προς τα πεδία Year και Vict_Descent. Για κάθε συνδυασμό αυτών των πεδίων υπολογίζεται το πλήθος των περιστατικών, ενώ τα αποτελέσματα ταξινομούνται κατά φθίνουσα σειρά ως προς το έτος. Για την κατάταξη των εγκλημάτων σύμφωνα με το πλήθος τους χρησιμοποιείται το window specification rank_window_spec, ενώ για τον υπολογισμό των συνολικών εγκλημάτων ανά έτος χρησιμοποιείται το total_window_spec. Με τη χρήση των παραπάνω προκύπτουν οι στήλες total_per_year, καθώς και η στήλη %, στην οποία αποτυπώνεται το ποσοστό των ετήσιων εγκλημάτων που αντιστοιχεί στη φυλετική ομάδα της κάθε γραμμής, και η στήλη rank, η οποία διατηρεί την ετήσια κατάταξη κάθε φυλετικής ομάδας σύμφωνα με τη συνάρτηση row_number() εντός του rank_window_spec. Στη συνέχεια, τα αποτελέσματα φιλτράρονται ώστε να εμφανίζονται μόνο οι τρεις φυλετικές ομάδες που ζητούνται. Τέλος, πραγματοποιείται αντιστοίχιση των κωδικών των φυλετικών ομάδων με τη λεπτομερή περιγραφή τους, η οποία περιέχεται στο DataFrame REcodes_df στη στήλη Vict_Descent_Full.

Υλοποίηση με SQL API

Για την υλοποίηση του ζητούμενου με χρήση του SQL API ακολουθήθηκε όσο το δυνατόν πιο κοντινή προσέγγιση στην αντίστοιχη υλοποίηση με το DataFrame API, ώστε οι χρονικές μετρήσεις που ζητούνται να είναι συγκρίσιμες. Η βασικότερη διαφορά μεταξύ των δύο μεθόδων εντοπίζεται στον τρόπο έκφρασης και διατύπωσης της κοινής τους λογικής, καθώς στο SQL API η επεξεργασία γίνεται μέσω SQL ερωτημάτων και προσωρινών views, ενώ στο DataFrame API χρησιμοποιούνται συναρτήσεις της βιβλιοθήκης Spark.

Σχολιασμός Μετρήσεων

Στις δύο υλοποιήσεις παρατηρήθηκε αμελητέα διαφορά στον χρόνο εκτέλεσης, γεγονός που αιτιολογείται από το ότι και στις δύο περιπτώσεις χρησιμοποιείται ο ίδιος optimizer. Πιθανές μικρές αποκλίσεις μπορεί να οφείλονται, όπως αναφέρθηκε και προηγουμένως, στον διαφορετικό τρόπο διατύπωσης και έκφρασης της λογικής της υλοποίησης σε κάθε API. Σημαντικό είναι να σημειωθεί ότι τα κελιά όπου πραγματοποιήθηκαν οι μετρήσεις χρόνου εκτελέστηκαν μετά από επανεκκίνηση του kernel (restarted kernel), ώστε να αποφευχθούν επιρροές από δεδομένα που είχαν ήδη υπολογιστεί και ήταν αποθηκευμένα στην cache.

Ζητούμενο 3

Υλοποίηση με RDDs

Στην υλοποίηση του ζητούμενου 3 με χρήση RDDs, χρησιμοποιείται το SparkContext για την ανάγνωση και επεξεργασία των δεδομένων εγκληματικότητας από τα δύο αρχεία CSV που καλύπτουν τις περιόδους 2010–2019 και 2020–2025, καθώς και από το αρχείο MO_codes.txt, το οποίο περιέχει την περιγραφή των MO codes. Όπως και στο ζητούμενο 1, τα αρχεία εγκληματικότητας διαβάζονται ως RDDs γραμμών, γίνεται ανάλυση CSV και από κάθε εγγραφή διατηρούνται το αναγνωριστικό του περιστατικού και το πεδίο που περιέχει τους MO codes. Στη συνέχεια, τα δύο RDDs ενοποιούνται σε ένα ενιαίο RDD και αφαιρείται η γραμμή επικεφαλίδας. Αντίστοιχα, το αρχείο MO_codes.txt διαβάζεται επίσης ως RDD και μετασχηματίζεται σε ζεύγη (MO code, περιγραφή), διαχωρίζοντας τον κωδικό από την περιγραφή με βάση το πρώτο κενό χαρακτήρα.

Για την καταμέτρηση των εμφανίσεων των MO codes, εφαρμόζεται flatMap στο RDD των εγκληματικών περιστατικών, ώστε κάθε εγγραφή να παράγει πολλαπλά ζεύγη της μορφής (MO code, 1), ένα για κάθε κωδικό που περιέχεται στο αντίστοιχο πεδίο. Στη συνέχεια, η reduceByKey χρησιμοποιείται για την άθροιση των εμφανίσεων κάθε MO code στο σύνολο των δεδομένων. Το αποτέλεσμα της καταμέτρησης συνενώνεται (join) με το RDD των περιγραφών των MO codes, αξιοποιώντας το κοινό κλειδί, ώστε κάθε κωδικός να συνοδεύεται από την περιγραφή του.

Τέλος, τα δεδομένα αναδιαμορφώνονται ώστε να περιέχουν τον MO code, την περιγραφή του και το πλήθος εμφανίσεων, ταξινομούνται κατά φθίνουσα σειρά με βάση τη συχνότητα εμφάνισης και τυπώνονται στην οθόνη. Παράλληλα, καταγράφεται ο συνολικός χρόνος εκτέλεσης, χωρίς να λαμβάνεται υπόψη η φόρτωση και η προεπεξεργασία των δεδομένων, ο οποίος ανέρχεται σε 26.29s.

Υλοποίηση με Dataframes API

Για υλοποίηση με χρήση του DataFrame API, όπως και στα παραπάνω ερωτήματα, ορίζεται ρητά το σχήμα των δεδομένων για τα εγκλήματα των περιόδων 2010–2019 και 2020–2025, φορτώνονται τα CSV αρχεία, διατηρούνται τα απαραίτητα πεδία και τα δύο dataframes ενοποιούνται με union. Παράλληλα, το αρχείο MO_codes.txt διαβάζεται σε ξεχωριστό DataFrame και μετασχηματίζεται, με χρήση της συνάρτησης split(), ώστε κάθε γραμμή να διαχωρίζεται σε δύο στήλες: τον κωδικό MO και την αντίστοιχη περιγραφή του.

Έπειτα, επειδή το πεδίο Mocodes του DataFrame εγκλημάτων περιέχει πολλαπλούς MO codes σε μορφή μιας ενιαίας συμβολοσειράς, απαιτείται μια διαδικασία λειτουργικά ισοδύναμη με τη χρήση της flatMap στην υλοποίηση με RDDs. Για τον λόγο αυτό, αρχικά εφαρμόζεται η συνάρτηση split, η οποία μετατρέπει τη συμβολοσειρά σε πίνακα επιμέρους κωδικών. Στη συνέχεια, η συνάρτηση explode χρησιμοποιείται ώστε κάθε στοιχείο του πίνακα να «ξεδιπλωθεί» σε ξεχωριστή γραμμή του DataFrame. Με αυτόν τον μετασχηματισμό, κάθε MO code εμφανίζεται πλέον ως ανεξάρτητη εγγραφή και μπορεί να επεξεργαστεί μεμονωμένα.

Ακολουθεί ομαδοποίηση των δεδομένων ως προς τον MO code με χρήση της groupBy και καταμέτρηση των εμφανίσεων μέσω της count. Τα αποτελέσματα συνενώνονται (join) με το DataFrame των περιγραφών των MO codes, αξιοποιώντας το κοινό πεδίο του κωδικού, ώστε κάθε καταμέτρηση να συνοδεύεται από την αντίστοιχη περιγραφή. Τέλος, τα αποτελέσματα ταξινομούνται κατά φθίνουσα σειρά με βάση το πλήθος εμφανίσεων και τυπώνονται στην οθόνη. Παράλληλα, καταγράφεται ο συνολικός χρόνος εκτέλεσης της υπολογιστικής διαδικασίας, ο οποίος ανέρχεται σε 8.40 s, σημαντικά μικρότερος σε σύγκριση με τον αντίστοιχο χρόνο εκτέλεσης της υλοποίησης με RDDs, γεγονός αναμενόμενο, όπως τεκμηριώθηκε και στο ζητούμενο 1.

Για να εξεταστεί το execution plan που επέλεξε ο Spark, χρησιμοποιήθηκε η εντολή explain. Από το πλάνο προκύπτει ότι ο Spark επέλεξε Broadcast Hash Join, γεγονός που υποδηλώνει ότι το DataFrame που περιέχει τους MO codes θεωρήθηκε μικρό σε μέγεθος και μεταδόθηκε σε όλους τους executors. Με τον τρόπο αυτό, το join εκτελείται τοπικά σε κάθε executor χωρίς εκτεταμένο shuffle των δεδομένων εγκληματικότητας, μειώνοντας σημαντικά το υπολογιστικό κόστος και βελτιώνοντας την απόδοση της διαδικασίας.

Χρησιμοποιώντας την εντολή `hint()`, εξαναγκάζουμε το Spark να χρησιμοποιήσει διαφορετικές στρατηγικές join και συγκεκριμένα `merge`, `shuffle_hash`, `shuffle_replicate_n1` και τα αποτελέσματα σε χρόνο εκτέλεσης φαίνονται παρακάτω:

Είδος join	Χρόνος εκτέλεσης
Broadcast Join	8.40s
Merge Join	11.34s
Shuffle Hash Join	10.61s
Shuffle Replicate Join	13.57s

Το Broadcast Join παρουσίασε τον μικρότερο χρόνο εκτέλεσης (8.40 s), γεγονός αναμενόμενο, καθώς αυτή είναι και η στρατηγική που επέλεξε ο optimizer. Όπως αναλύθηκε και παραπάνω, η συγκεκριμένη προσέγγιση είναι ιδανική σε περιπτώσεις όπου τα datasets έχουν τόσο διαφορετικά μεγέθη. Το Shuffle Hash Join ακολούθησε με χρόνο εκτέλεσης 10.61 s, καθώς απαιτεί ανακατανομή (shuffle) των δεδομένων ώστε εγγραφές με το ίδιο κλειδί να καταλήξουν στο ίδιο partition. Το Merge Join εμφάνισε ελαφρώς μεγαλύτερο χρόνο εκτέλεσης (11.34 s), καθώς, πέρα από το shuffle, απαιτεί και ταξινόμηση των δεδομένων και στις δύο πλευρές του join, γεγονός που αυξάνει το υπολογιστικό κόστος. Τέλος, το Shuffle Replicate Join ήταν το πιο αργό (13.57 s), καθώς βασίζεται σε μηχανισμό nested loop, όπου η μικρή πλευρά του join αναπαράγεται σε κάθε partition της μεγάλης πλευράς, οδηγώντας σε αυξημένο αριθμό συγκρίσεων, αν και στο συγκεκριμένο σενάριο το κόστος αυτό περιορίστηκε λόγω του πολύ μικρού μεγέθους του πίνακα των MO codes.

Ζητούμενο 4

Υλοποίηση με Dataframes API

Για υλοποίηση με χρήση του DataFrame API, όπως και στα παραπάνω ερωτήματα, ορίστηκε ρητά το σχήμα των δεδομένων για τα εγκλήματα των περιόδων 2010–2019 και 2020–2025, φορτώθηκαν τα CSV αρχεία, διατηρήθηκαν τα απαραίτητα πεδία, ενοποιήθηκαν με `union` τα 2 dataframes και απορρίφθηκαν εγγραφές με μη έγκυρες ή μηδενικές συντεταγμένες. Αντίστοιχα, διαβάστηκαν τα δεδομένα των αστυνομικών τμημάτων, από τα οποία διατηρήθηκαν οι γεωγραφικές συντεταγμένες και τα divisions.

Στη συνέχεια, οι συντεταγμένες μετατράπηκαν σε γεωμετρικά σημεία με τη χρήση της συνάρτησης `ST_Point`. Αξίζει να σημειωθεί ότι χρησιμοποιήθηκε η συνάρτηση `expr()` του Spark SQL για την κλήση των χωρικών συναρτήσεων, καθώς επιτρέπει την

ενσωμάτωση SQL εκφράσεων στο DataFrame API και απέφυγε το σφάλμα 'JavaPackage' object is not callable που προέκυπτε.

Για τον εντοπισμό του πλησιέστερου αστυνομικού τμήματος σε κάθε έγκλημα, πραγματοποιήθηκε crossJoin μεταξύ των DataFrames εγκλημάτων και σταθμών, δημιουργώντας όλα τα δυνατά ζεύγη. Για κάθε ζεύγος υπολογίστηκε η απόσταση με τη συνάρτηση ST_DistanceSphere, η οποία επιστρέφει την απόσταση σε μέτρα και στη συνέχεια μετατράπηκε σε χιλιόμετρα. Έπειτα, χρησιμοποιήθηκε window function με ομαδοποίηση ως προς το DR_NO και ταξινόμηση με βάση την απόσταση, ώστε να επιλεγεί για κάθε έγκλημα το κοντινότερο αστυνομικό τμήμα.

Τέλος, τα αποτελέσματα ομαδοποιήθηκαν ανά division και υπολογίστηκαν τόσο ο μέσος όρος της απόστασης των εγκλημάτων από τα πλησιέστερα αστυνομικά τμήματα όσο και το πλήθος των εγκλημάτων ανά division. Τα αποτελέσματα ταξινομήθηκαν κατά φθίνουσα σειρά με βάση τον αριθμό των εγκλημάτων και τυπώθηκαν στην οθόνη.

Αξίζει να σημειωθεί πως, από το explain(formatted) φαίνεται ότι ο optimizer του Spark επιλέγει για το join μεταξύ εγκλημάτων και αστυνομικών τμημάτων τη στρατηγική Broadcast Nested Loop Join (Cross, BuildRight). Αυτό είναι αναμενόμενο, επειδή στο query χρησιμοποιείται CROSS JOIN, άρα δεν μπορούν να εφαρμοστούν στρατηγικές όπως BroadcastHashJoin ή SortMergeJoin και έτσι η κατάλληλη επιλογή είναι nested loop join. Παράλληλα, ο Spark αποφασίζει να κάνει broadcast το DataFrame των σταθμών (δεξιά πλευρά / BuildRight), όπως φαίνεται από το BroadcastExchange, επειδή είναι πολύ μικρό σε μέγεθος σε σχέση με τα δεδομένα εγκλημάτων. Έτσι, το μικρό dataset αντιγράφεται σε όλους τους executors και ο υπολογισμός των αποστάσεων (ST_DistanceSphere) γίνεται τοπικά για κάθε έγκλημα, αποφεύγοντας μεγάλο shuffle εξαιτίας του join. Στη συνέχεια, η επιλογή του πλησιέστερου σταθμού ανά DR_NO υλοποιείται με window operator (row_number) και εμφανίζονται στάδια Sort και Exchange (hashpartitioning σε DR_NO) ώστε να ικανοποιηθούν οι απαιτήσεις ταξινόμησης/κατανομής των δεδομένων για το window, ενώ ακολουθεί το τελικό aggregation ανά DIVISION με δύο φάσεις HashAggregate και shuffle, καθώς και τελική ταξινόμηση για την κατάταξη κατά num_crimes

Υλοποίηση με SQL API

Για λόγους πληρότητας, το ζητούμενο 4 υλοποιήθηκε και με χρήση του SQL API του Spark. Η βασική διαφορά σε σχέση με την υλοποίηση με DataFrames έγκειται στον τρόπο έκφρασης της λογικής, καθώς οι ίδιες πράξεις (cross join, υπολογισμός απόστασης, επιλογή του πλησιέστερου αστυνομικού τμήματος μέσω window function και τελική ομαδοποίηση) υλοποιούνται αποκλειστικά με SQL ερωτήματα πάνω σε προσωρινά views. Συγκεκριμένα, ο υπολογισμός της απόστασης και η επιλογή του κοντινότερου τμήματος πραγματοποιούνται με χρήση CROSS JOIN και ROW_NUMBER() σε SQL, αντί των αντίστοιχων DataFrame transformations, ενώ το

τελικό aggregation εκφράζεται με GROUP BY και ORDER BY. Κατά τα λοιπά, η υπολογιστική λογική και τα παραγόμενα αποτελέσματα παραμένουν ίδια με την υλοποίηση μέσω DataFrame API.

Σε αυτό το σημείο, υπογραμμίζεται ότι το Data Frame API και το SQL API χρησιμοποιούν τον ίδιο optimizer, γεγονός που σημαίνει ότι παρατηρείται ίδιο execution plan και παρόμοιοι χρόνοι εκτέλεσης.

Σχολιασμός Αποτελεσμάτων για Διαφορετικά Configurations

Configurations	Χρόνος Εκτέλεσης
1 core, 2 GB memory	38.65s
2 cores, 4GB memory	28.86s
4 cores, 8GB memory	22.66s

Από τα αποτελέσματα του ερωτήματος 4 προκύπτει ότι η αύξηση των διαθέσιμων υπολογιστικών πόρων οδηγεί σε σταδιακή βελτίωση της απόδοσης. Η μείωση του χρόνου εκτέλεσης όσο αυξάνονται οι cores και η διαθέσιμη μνήμη υποδεικνύει ότι το συγκεκριμένο ερώτημα επωφελείται από την αυξημένη παραλληλία, κυρίως λόγω της εκτέλεσης απαιτητικών πράξεων όπως το cross join, οι υπολογισμοί αποστάσεων και τα shuffle operations. Η παρατηρούμενη συμπεριφορά αντιστοιχεί σε scale-up βελτίωση, καθώς η επιτάχυνση επιτυγχάνεται μέσω της ενίσχυσης των πόρων στον ίδιο κόμβο και όχι μέσω κατανομής του φόρτου σε περισσότερους κόμβους (scale-out). Παρότι η αύξηση των πόρων βελτιώνει την απόδοση, η κλιμάκωση δεν είναι γραμμική, γεγονός που υποδηλώνει την ύπαρξη εγγενών περιορισμών που σχετίζονται με το κόστος συγχρονισμού και τις βαριές φάσεις επεξεργασίας του ερωτήματος.

Ζητούμενο 5

Εύρεση συσχέτισης Income per Capita και Crime per Capita

Για να βρεθεί η ζητούμενη συσχέτιση με την ελάχιστη δυνατή απώλεια πληροφορίας από τα δεδομένα έγινε πρώτα equijoin μεταξύ των datasets LA_income και LA_census_blocks στη στήλη Zip Codes και ZCTA20 αντίστοιχα για το καθένα. Γίνεται επομένως αντιστοίχιση των blocks με τα incomes των περιοχών στις οποίες ανήκουν και υπολογίζεται το income per block. Ύστερα, γίνεται spatial join των crimes και των blocks έτσι ώστε να τοποθετηθούν τα εγκλήματα στα blocks που έγιναν. Εκτελείται group by geometry και count() έτσι ώστε να προκύψουν τα συνολικά εγκλήματα ανά block. Τέλος, γίνεται η ομαδοποίηση ανά COMM ,

υπολογίζεται ο συνολικός πληθυσμός, το συνολικό εισόδημα και τα συνολικά εγκλήματα ανά block. Δημιουργούνται οι στήλες `income per capita` και `crimes per person` στις οποίες καλείται η συνάρτηση `corr` και υπολογίζεται η συσχέτιση Pearson.

Η συσχέτιση που προκύπτει για όλα τα communities είναι 0.117 και για τα 10 communities με το υψηλότερο και χαμηλότερο εισόδημα -0.425. Τα αποτελέσματα αυτά οφείλονται στη δομή των ίδιων των communities και των blocks που τα αποτελούν στα οποία εμφανίζεται μεγάλη απόκλιση στην εγκληματικότητα και το μέσο εισόδημα. Επιπλέον, η ελαφρώς μικρότερη συσχέτιση που παρατηρείται μπορεί να οφείλεται στη χρήση των ZCTA20 και Zip Code στηλών για τις οποίες δεν υπάρχει τέλεια αντιστοιχία οδηγώντας στον αποκλεισμό κάποιων εγκλημάτων από τη μέτρηση. Χαρακτηριστικό παράδειγμα είναι το ZCTA20 90089 που ενώ στα census blocks αντιπροσωπεύει μια περιοχή με 913 κατοίκους δεν υπάρχει η αντίστοιχη εγγραφή στο income dataset. Τέλος, η απόσταση των μετρήσεων του correlation στις δύο περιπτώσεις είναι λογική και αναμενόμενη, καθώς οι περιοχές με το υψηλότερο και χαμηλότερο εισόδημα απέχουν σημαντικά στην εγκληματικότητα αλλά και δεν εμφανίζονται σε αυτές το ίδιο έντονα προβλήματα ομαδοποίησης blocks που απέχουν σημαντικά στα μετρούμενα μεγέθη.

Σχολιασμός των Cases

Τα αποτελέσματα των cases 1,2,3 που θα αναλυθούν παρακάτω αφορούν τα joins που υλοποιήθηκαν και προκύπτουν από την εκτέλεση του κώδικα μετά από επανεκκίνηση του session έτσι ώστε να μπορεί να γίνει ορθότερος σχολιασμός και να μη ληφθούν υπόψη τα cached δεδομένα.

Join blocks και income

Παρατηρούμε ότι ο optimizer επιλέγει broadcast hash inner join, καθώς το LA_income είναι σημαντικά μικρότερο σε μέγεθος από το LA_blocks και επομένως μπορεί να μεταφερθεί αποδοτικά στους executors. Ο χρόνος εκτέλεσης του join είναι κοντινός και στα τρία cases, με την καλύτερη απόδοση να παρατηρείται στο case 1. Στο case 1, η καλύτερη επίδοση οφείλεται στο ότι το broadcasted dataframe και τα αντίστοιχα hash maps υλοποιούνται μία φορά ανά executor και επαναχρησιμοποιούνται από πολλαπλά tasks, γεγονός που μειώνει το κόστος broadcast, βελτιώνει την τοπικότητα μνήμης και περιορίζει το overhead από serialization. Αντίθετα, στο case 2 παρατηρείται η χειρότερη επίδοση, καθώς συνδυάζεται αυξημένο κόστος broadcast λόγω περισσότερων executors με περιορισμένη επαναχρησιμοποίηση των hash δομών, οδηγώντας σε μεγαλύτερο συνολικό overhead χωρίς ουσιαστικό όφελος σε παραλληλισμό. Στο case 3, η απόδοση είναι ελαφρώς καλύτερη από το case 2, λόγω απλούστερης εκτέλεσης tasks και απουσίας ανταγωνισμού μεταξύ cores εντός του ίδιου executor, ωστόσο παραμένει κατώτερη του case 1 εξαιτίας της ανάγκης αντιγραφής των broadcast δεδομένων σε περισσότερους executors και της έλλειψης

επαναχρησιμοποίησης των hash tables. Συνολικά, τα δεδομένα είναι αρκετά μικρά ώστε τα partitions του LA_blocks, το broadcasted dataframe, οι εσωτερικές δομές του συστήματος και τα hash maps να χωρούν στη μνήμη χωρίς σημαντικά memory spills και I/Os, γεγονός που εξηγεί τις σχετικά μικρές διαφορές χρόνου μεταξύ των cases. Επιπλέον, οι πολύ μικροί executors αυξάνουν τη συνολική κατανάλωση μνήμης για κοινά δεδομένα και περιορίζουν τον διαθέσιμο χώρο για τα YARN daemons και τον Application Master, επηρεάζοντας αρνητικά την αποδοτικότητα. Τέλος, ενώ η αύξηση του αριθμού των executors μπορεί να βελτιώσει αισθητά workloads όπως τα groupby queries όπου ο παραλληλισμός αξιοποιείται άμεσα, στα broadcast hash joins δεν οδηγεί σε αντίστοιχη βελτίωση της επίδοσης, καθώς κυρίαρχο ρόλο παίζει το κόστος του broadcast και η επαναχρησιμοποίηση των hash δομών.

Join Crimes και Blocks_and_income

Εδώ γίνεται range join για να βρεθούν τα overlapping geometries και να γίνουν join τα crimes με τα blocks στα οποία ανήκουν. Ο τρόπος που υλοποιείται το range join δεν γνωστοποιείται στο χρήστη μέσω του explain. Γνωρίζουμε ότι τα spatial joins είναι inner.